INFORME DE LABORATORIO

Observabilidad básica de microservicios con Spring Boot Actuator y Micrometer (sin Docker)

Asignatura: Sistemas Distribuidos

Dato	Detalle
Autor	Sebastian Montes Olivera
Fecha	31 de octubre de 2026
Tecnologías	Java 21, Spring Boot 3.5.10, Spring Boot Actuator, Micrometer, Maven
Repositorio	https://github.com/SebastianMontes-Dev/observabilidad-microservicios-taller

Contenido

1. Propósito y alcance 3
2. Entorno utilizado 3
2.1 Diferencias respecto a la guía 3
3. Microservicio de productos 4
3.1 Configuración 4
3.2 Controlador 4
4. Microservicio de pedidos 5
4.1 Configuración 5
4.2 Cliente HTTP 5
4.3 Controlador de pedidos 5
5. Ejecución simultánea 6
5.1 Ambos microservicios en ejecución 6
5.2 Respuesta de /pedidos/1 7
6. Exploración de Actuator 7
7. Experimento 1: métricas de solicitudes 8
7.1 Indicadores 8
7.2 Tabla antes / después (filtrada por URI) 8
7.3 Tabla antes / después (agregada, sin filtrar) 9
7.4 Interpretación 9
8. Experimento 2: logs y fallo de dependencia 11
8.1 Logs con ambos servicios activos 11
8.2 Con productos detenido 11
8.3 ¿Permanece UP el health de pedidos? 12
8.4 Recuperación 12
9. Preguntas de análisis 12
10. Conclusión 13
11. Repositorio y reproducción 14

1. Propósito y alcance

Se implementaron dos microservicios REST independientes, se observaron sus indicadores de salud y sus métricas HTTP, se examinaron los registros (logs) y se analizó el efecto de que una dependencia remota deje de estar disponible.

- Construir y ejecutar dos aplicaciones Spring Boot de forma simultánea.
- Exponer y consultar los endpoints de Spring Boot Actuator.
- Interpretar COUNT, TOTAL_TIME y MAX en las métricas HTTP.
- Identificar logs de negocio y diferenciar la salud local de la disponibilidad de las dependencias.

Componente	Rol	Puerto
microservicio-productos	Devuelve los datos de un producto (JSON)	8081
microservicio-pedidos	Consulta un pedido y obtiene el producto llamando por HTTP a productos	8082

Flujo: Cliente HTTP → microservicio-pedidos (8082) → microservicio-productos (8081). Ambos publican /actuator/health y /actuator/metrics. No se usó Docker, MySQL, Prometheus ni Grafana.

2. Entorno utilizado

Elemento	Versión / detalle
Sistema operativo	Windows 11 Pro
JDK	Eclipse Temurin 21.0.12
Maven	Maven Wrapper (mvnw) con Apache Maven 3.9.11
Spring Boot	3.5.10 (rama 3.5.x que pide la guía)
Dependencias	Spring Web y Spring Boot Actuator
IDE	Visual Studio Code con Extension Pack for Java y Spring Boot Dashboard
Navegador	Microsoft Edge
Control de versiones	Git / GitHub

2.1 Diferencias respecto a la guía

- **IDE:** la guía usa IntelliJ IDEA; se empleó Visual Studio Code. Las capturas de "ejecución en el IDE" corresponden al Spring Boot Dashboard y a la terminal de depuración de VS Code. Los proyectos son Maven estándar y se importan sin cambios en IntelliJ.
- **Spring Boot 3.5.10:** start.spring.io ya solo ofrece la rama 4.x, así que los proyectos se generaron allí para obtener el Maven Wrapper y el pom.xml se fijó a Spring Boot 3.5.10 con spring-boot-starter-web, tal como pide la guía.

- **Archivo .http:** pruebas-observabilidad.http se entrega tal cual en microservicio-pedidos; en VS Code se puede ejecutar con la extensión REST Client y en IntelliJ con su cliente HTTP.
- **Parada de productos:** en el experimento 2 el servicio de productos se detuvo terminando el proceso que escucha en el puerto 8081 (equivale al paso de "detener solo microservicio-productos").

3. Microservicio de productos

Campo	Valor
Group / Artifact	com.ejemplo / microservicio-productos
Package	com.ejemplo.productos
Java / Packaging	21 / Jar
Dependencias	Spring Web, Spring Boot Actuator

3.1 Configuración

src/main/resources/application.properties

```
spring.application.name=microservicio-productos
server.port=8081
management.endpoints.web.exposure.include=health,info,metrics
management.endpoint.health.show-details=always
```

3.2 Controlador

src/main/java/com/ejemplo/productos/controller/ProductoController.java

```
package com.ejemplo.productos.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.*;
import java.util.Map;

@RestController
@RequestMapping("/productos")
public class ProductoController {

    private static final Logger log = LoggerFactory.getLogger(ProductoController.class);

    @GetMapping("/{id}")
    public Map<String, Object> obtener(@PathVariable Long id) {

        log.info("Consultando producto con ID {}", id);
        return Map.of("id", id, "nombre", "Computador portatil", "precio", 2500000);
    }
}
```

El controlador registra un mensaje de negocio por cada consulta y devuelve un Map con id, nombre y precio. Esa línea de log es la que se busca luego en la consola.

4. Microservicio de pedidos

Campo	Valor
Group / Artifact	com.ejemplo / microservicio-pedidos
Package	com.ejemplo.pedidos
Java / Packaging	21 / Jar
Dependencias	Spring Web, Spring Boot Actuator

4.1 Configuración

src/main/resources/application.properties

```
spring.application.name=microservicio-pedidos
server.port=8082
management.endpoints.web.exposure.include=health,info,metrics
management.endpoint.health.show-details=always
```

4.2 Cliente HTTP

src/main/java/com/ejemplo/pedidos/RestClientConfig.java

```
package com.ejemplo.pedidos;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

@Configuration
public class RestClientConfig {

    @Bean
    public RestClient restClient(RestClient.Builder builder) {

        return builder.build();
    }
}
```

4.3 Controlador de pedidos

src/main/java/com/ejemplo/pedidos/controller/PedidoController.java

```
package com.ejemplo.pedidos.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.client.RestClient;
import java.util.Map;

@RestController
@RequestMapping("/pedidos")
public class PedidoController {

    private static final Logger log = LoggerFactory.getLogger(PedidoController.class);
    private final RestClient restClient;

    public PedidoController(RestClient restClient) {
```

```
        this.restClient = restClient;
    }

    @GetMapping("/{id}")
    public Map<String, Object> consultar(@PathVariable Long id) {

        log.info("Iniciando consulta del pedido {}", id);
        Map<?, ?> producto = restClient.get()
                .uri("http://localhost:8081/productos/{id}", id)
                .retrieve()
                .body(Map.class);
        log.info("Consulta del producto finalizada para pedido {}", id);
        return Map.of("pedidoId", id, "estado", "PROCESADO", "producto", producto);
    }
}
```

Para simplificar el taller se usa Map como representación de datos. En un proyecto real conviene definir DTO tipados, validación, timeout y manejo de errores; precisamente la ausencia de timeout y de manejo de errores es lo que hace visible el fallo del experimento 2.

5. Ejecución simultánea

Se inició primero microservicio-productos (8081) y después microservicio-pedidos (8082) desde el Spring Boot Dashboard de VS Code, sin detener el primero. Tras consultar /pedidos/1 aparecen registros de consulta en ambas consolas.

5.1 Ambos microservicios en ejecución

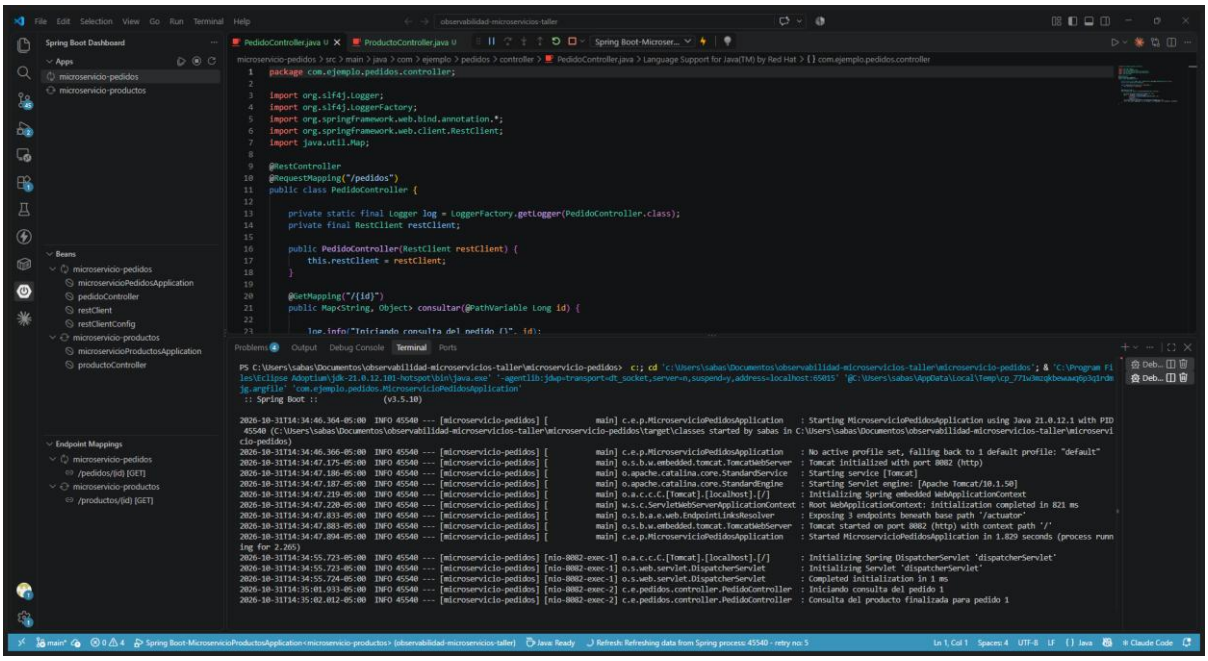


Figura 1. VS Code: Spring Boot Dashboard con ambas aplicaciones y la consola de microservicio-pedidos (puerto 8082) con sus logs de negocio.

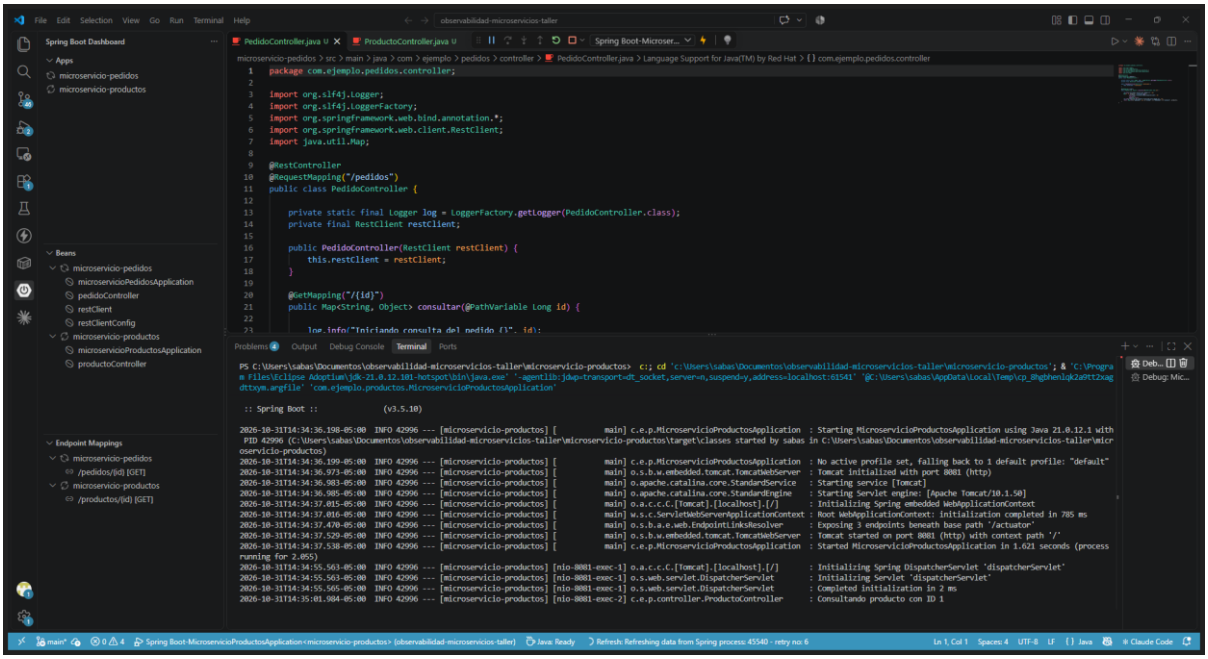


Figura 2. VS Code: consola de microservicio-productos (puerto 8081) con la línea "Consultando producto con ID 1".

5.2 Respuesta de /pedidos/1

Resultado esperado y obtenido: JSON con `pedidoId=1`, `estado=PROCESADO` y el objeto `producto` anidado.

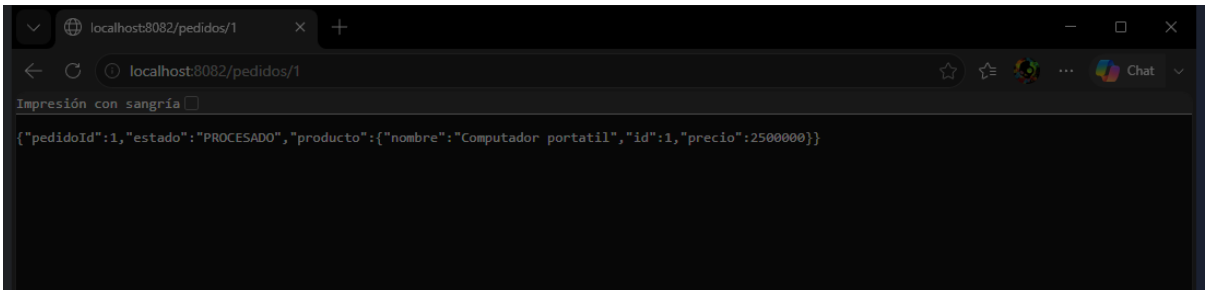


Figura 3. GET http://localhost:8082/pedidos/1: respuesta JSON de pedidos con el producto anidado.

Ejecución verificada también con pruebas automáticas: mvnw test en cada módulo (productos: 2 pruebas; pedidos: 3 pruebas, con el servicio de productos simulado) y el script docs/scripts/ejecutar-experimentos.ps1.

6. Exploración de Actuator

URL	Qué se observa
`http://localhost:8081/actuator/health`	Salud del servicio de productos
`http://localhost:8082/actuator/health`	Salud del servicio de pedidos
`http://localhost:8082/actuator/metrics`	Nombres de las métricas disponibles

URL	Qué se observa
`http://localhost:8082/actuator/metrics/http.server.requests`	Mediciones HTTP de pedidos
`http://localhost:8081/actuator/metrics/http.server.requests`	Mediciones HTTP de productos

Con `management.endpoint.health.show-details=always` el health muestra sus componentes (diskSpace, ping y ssl). Ambos servicios responden `"status":"UP"`.

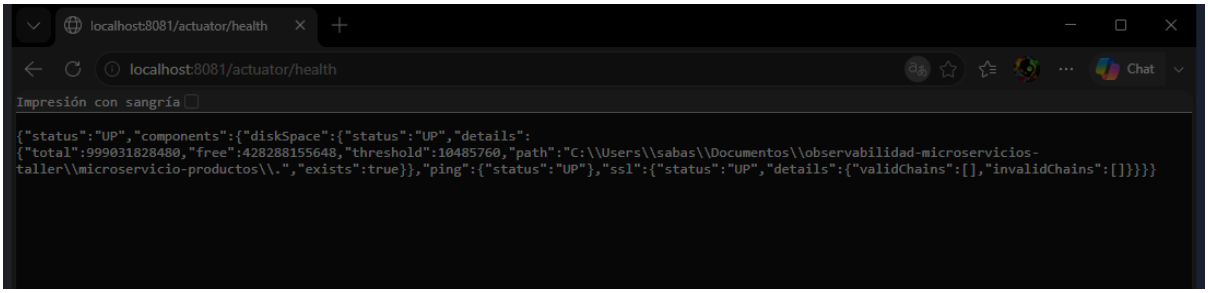


Figura 4. GET http://localhost:8081/actuator/health: microservicio-productos en estado UP.

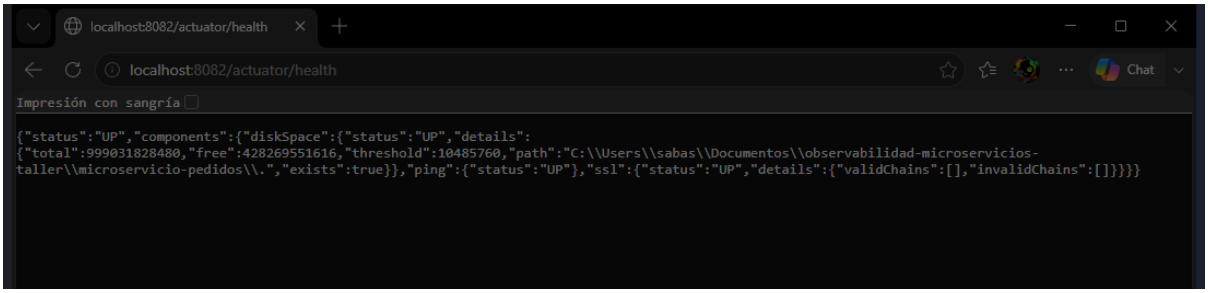


Figura 5. GET http://localhost:8082/actuator/health: microservicio-pedidos en estado UP.

7. Experimento 1: métricas de solicitudes

Procedimiento: se consultó una vez /pedidos/1, se anotaron COUNT, TOTAL_TIME y MAX de http.server.requests, se hicieron diez solicitudes adicionales y se comparó. Después se repitió la consulta de métricas en productos (8081).

7.1 Indicadores

Indicador	Significado
`COUNT`	Número de peticiones registradas en la serie consultada
`TOTAL_TIME`	Suma de la duración registrada, en segundos (baseUnit = seconds)
`MAX`	Tiempo máximo observado en la ventana del medidor

7.2 Tabla antes / después (filtrada por URI)

Para no mezclar las llamadas a Actuator con las de negocio se filtró la serie con `?tag=uri:/pedidos/{id}` (y `/productos/{id}` en el otro servicio). Tiempos en segundos.

Servicio / URI	Momento	COUNT	TOTAL_TIME (s)	MAX (s)	Media (s)
pedidos /pedidos/{id}	Antes (1 consulta)	1	0,0852	0,0852	0,0852
pedidos /pedidos/{id}	Después (+10)	11	0,1418	0,0852	0,0129
productos /productos/{id}	Antes (1 consulta)	1	0,0083	0,0083	0,0083
productos /productos/{id}	Después (+10)	11	0,0231	0,0083	0,0021

7.3 Tabla antes / después (agregada, sin filtrar)

Es lo que devuelve la URL sin filtros, transcrito de las capturas. Incluye las propias consultas a /actuator y peticiones 404 del navegador (/favicon.ico), por eso los COUNT no son 1 y 11.

Servicio	Momento	COUNT	TOTAL_TIME (s)	MAX (s)
pedidos (8082)	Antes	3	0,1841	0,0852
pedidos (8082)	Después	18	0,2762	0,0852
productos (8081)	Antes	3	0,0929	0,0652
productos (8081)	Después	21	0,1935	0,0652

7.4 Interpretación

- **COUNT:** en pedidos pasó de 1 a 11 (+10) y en productos de 1 a 11 (+10): cada petición a pedidos provoca exactamente una a productos, por eso ambos contadores crecen igual.
- **TOTAL_TIME:** es acumulativo. En pedidos subió 0,0566 s con 10 peticiones nuevas (≈ 0,0057 s cada una), y en productos 0,0148 s (≈ 0,0015 s cada una). Pedidos tarda más porque su tiempo incluye la llamada HTTP a productos.
- **MAX:** no cambió (0,0852 s en pedidos y 0,0083 s en productos): lo marcó la primera petición, más lenta por la inicialización perezosa de clases, serialización JSON y conexión del cliente. Las diez siguientes fueron más rápidas, así que el máximo se conserva. La media (TOTAL_TIME / COUNT) baja de 0,0852 s a 0,0129 s en pedidos.
- **Agregado frente a filtrado:** la serie sin filtrar mezcla /actuator/health, /actuator/metrics, /pedidos/{id} y las rutas 404; para analizar un endpoint conviene filtrar por tag uri y revisar availableTags.



Figura 6. Métricas HTTP de pedidos, antes de las diez solicitudes (serie agregada).



Figura 7. Métricas HTTP de pedidos filtradas por uri=/pedidos/{id}, antes.



Figura 8. Métricas HTTP de productos, antes (serie agregada).



Figura 9. Métricas HTTP de pedidos, después de las diez solicitudes (serie agregada).



Figura 10. Métricas HTTP de pedidos filtradas por uri=/pedidos/{id}, después.



Figura 11. Métricas HTTP de productos, después (serie agregada).

8. Experimento 2: logs y fallo de dependencia

Procedimiento: con ambos servicios respondiendo bien se detuvo solo microservicio-productos (8081), se consultó /pedidos/1, se observó el error y el log de excepción, se consultó /actuator/health de pedidos y se reinició productos para comprobar la recuperación.

8.1 Logs con ambos servicios activos

Líneas de negocio de ambas consolas (ejecución por script, docs/evidencias/script/)

```
14:27:57.730 INFO  [pedidos] PedidoController: Iniciando consulta del pedido 1
14:27:57.794 INFO  [pedidos] PedidoController: Consulta del producto finalizada para pedido 1
14:27:57.656 INFO  [productos] ProductoController: Consultando producto con ID 1
```

8.2 Con productos detenido

La consulta `GET /pedidos/1` devuelve HTTP 500 (página de error de Spring Boot) y la consola de pedidos registra la excepción:

Línea de error en la consola de pedidos (ejecución por script)

```
14:28:08.606 ERROR [pedidos] [dispatcherServlet]: Servlet.service() for servlet [dispatcher…
```

La causa raíz es `java.nio.channels.ClosedChannelException` / conexión rechazada hacia `http://localhost:8081/productos/1`, envuelta en `ResourceAccessException`.

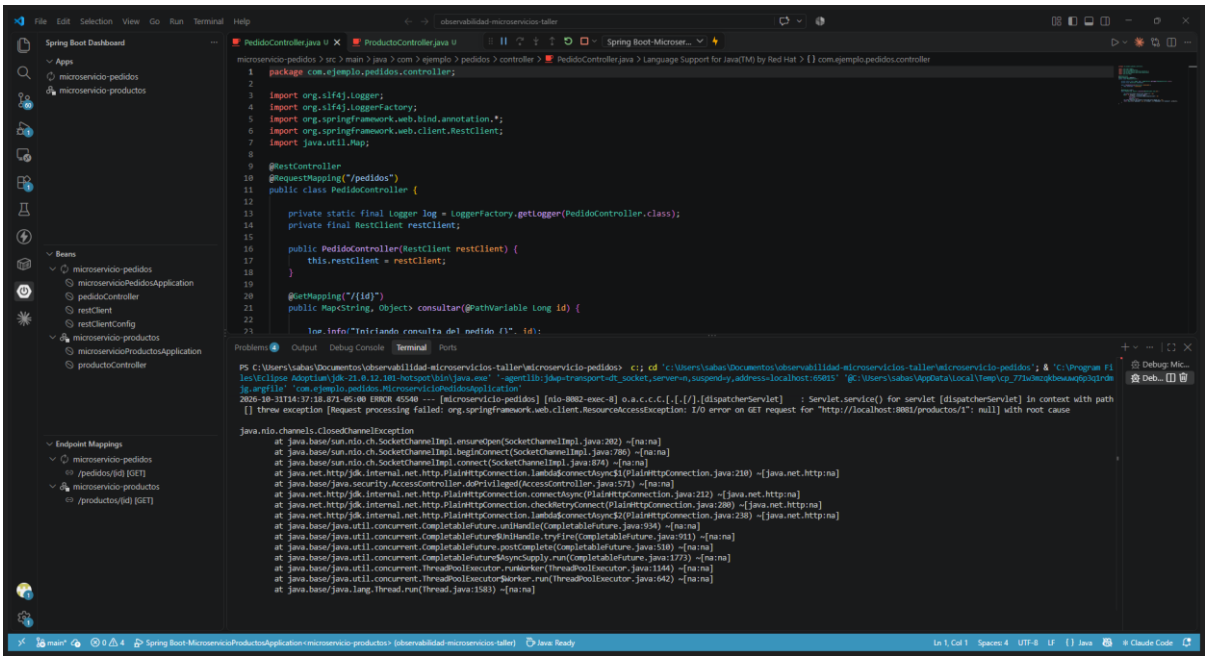


Figura 12. VS Code: consola de microservicio-pedidos con la excepción al estar productos detenido.

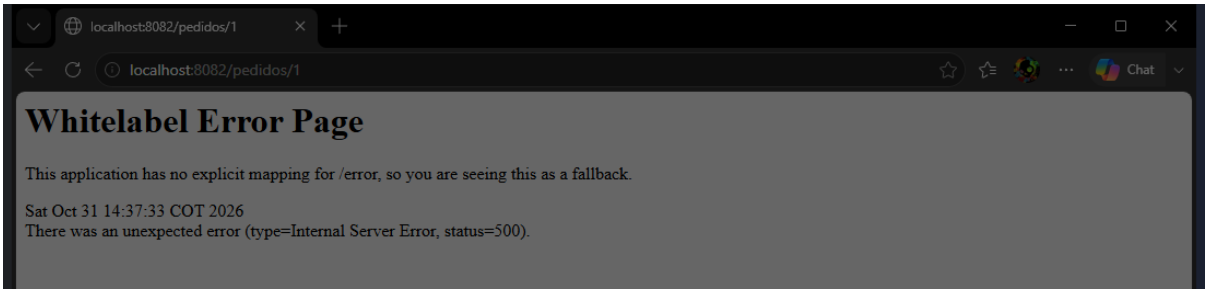


Figura 13. GET /pedidos/1 con productos detenido: HTTP 500.

8.3 ¿Permanece UP el health de pedidos?

Sí. Aunque la operación de negocio falla, /actuator/health de pedidos sigue devolviendo `"status":"UP"`. Es el comportamiento esperado y deliberado en el ejemplo: el health predeterminado solo comprueba indicadores locales (diskSpace, ping, ssl) y no sondea el servicio HTTP remoto.

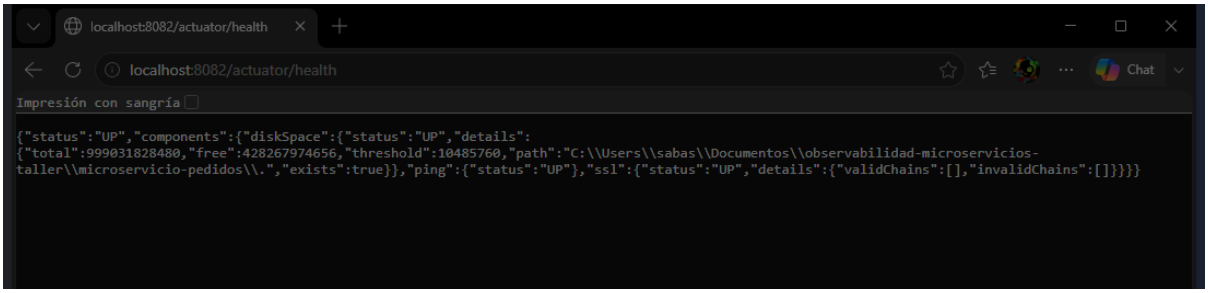


Figura 14. GET /actuator/health de pedidos con productos detenido: sigue en UP.

8.4 Recuperación

Al reiniciar microservicio-productos, pedidos vuelve a responder 200 con el JSON completo sin necesidad de reiniciarlo: no mantiene estado de la conexión caída.



Figura 15. GET /pedidos/1 tras reiniciar productos: respuesta normal.

9. Preguntas de análisis

1. ¿Qué diferencia existe entre COUNT y TOTAL_TIME?

COUNT es cuántas peticiones se han registrado; TOTAL_TIME es la suma de cuánto tardaron todas. Son acumulativos desde el arranque. Con ambos se obtiene la media (TOTAL_TIME / COUNT): en pedidos, 11 peticiones sumaron 0,1418 s, es decir 0,0129 s por petición. Un COUNT alto con TOTAL_TIME bajo significa tráfico rápido; un COUNT bajo con TOTAL_TIME alto, peticiones lentas.

2. ¿Por qué el servicio pedidos puede mostrar UP si productos está detenido?

Porque el health predeterminado de Spring Boot solo evalúa la salud del propio proceso (disco, ping, SSL). Saber si pedidos está "vivo" no es lo mismo que saber si puede cumplir su función. Para reflejar la dependencia habría que añadir un HealthIndicator propio que sondee a productos, o aplicar timeouts y un circuit breaker, y distinguir liveness de readiness.

3. ¿Qué datos de observabilidad permitirían detectar degradación de rendimiento?

- La media y el máximo de http.server.requests por URI (TOTAL_TIME / COUNT y MAX), y sobre todo percentiles (p95/p99) con histogramas habilitados.
- La tasa de respuestas con outcome SERVER_ERROR o status 5xx, y la serie de excepciones (tag exception).
- Métricas de la JVM y del servidor (hilos, memoria, GC, conexiones) y métricas del cliente HTTP hacia productos.
- Logs con duración por operación y, mejor, trazas que muestren en qué salto se pierde el tiempo.

4. ¿Qué aporta una traza distribuida que no da el log individual de cada servicio?

Una traza une, con un identificador común (traceId), todas las operaciones de una petición a través de varios servicios y muestra la jerarquía y la duración de cada tramo (span). El log de cada servicio solo ve su parte: hay que correlacionar a mano marcas de tiempo. Con la traza se ve directamente que "/pedidos/1 tardó X ms, de los cuales Y fueron la llamada a productos", o en qué servicio se originó el error. Queda como ampliación (Micrometer Tracing con Zipkin).

5. ¿Qué riesgos tiene exponer /actuator/metrics y /actuator/health con detalles en producción?

- Revelan información interna: rutas internas, versiones, espacio en disco y rutas del sistema de archivos (se vio el path local en diskSpace), nombres de beans, URIs y patrones de uso. Eso facilita el reconocimiento previo a un ataque.
- Sin autenticación, cualquiera puede consultarlos repetidamente (consumo de recursos) y deducir el estado del sistema.
- Mitigación: exponer solo lo necesario, mover Actuator a un puerto de gestión interno (management.server.port), protegerlo con Spring Security, usar show-details=when-authorized y limitar el acceso por red.

10. Conclusión

- **Métricas, logs y trazas.** Las métricas son números agregados en el tiempo (cuántas peticiones, cuánto tardan) y responden "¿qué está pasando en general?"; son baratas y buenas para alertar. Los logs son eventos puntuales con contexto y responden "¿qué pasó exactamente en este momento?". Las trazas siguen una petición concreta a través de varios servicios y responden "¿dónde se fue el tiempo o se produjo el fallo?". Se complementan: la métrica avisa, la traza localiza y el log explica.
- **Limitación del health predeterminado.** Solo refleja la salud local del proceso. Pedidos siguió en UP mientras todas sus peticiones fallaban con HTTP 500 porque su dependencia estaba caída; un orquestador o balanceador que se guíe solo por ese health seguiría enviándole tráfico. Hace falta incluir las dependencias críticas en un indicador de readiness, además de timeouts y manejo de errores.
- **Resultado.** Ambos microservicios se ejecutaron simultáneamente, se comprobó la comunicación HTTP, las métricas crecieron como se esperaba (COUNT 1 → 11 en la serie de

negocio), el fallo de la dependencia quedó registrado en logs y el servicio se recuperó al reiniciar productos.

- **Buenas prácticas pendientes.** Configuración académica y local: en producción restringir los endpoints de administración, no mostrar detalles de salud públicamente, establecer timeouts, instrumentar las dependencias y centralizar la recolección (Prometheus y Grafana, y Micrometer Tracing con Zipkin como siguiente práctica).

11. Repositorio y reproducción

Código, pruebas, evidencias y este informe: https://github.com/SebastianMontes-Dev/observabilidad-microservicios-taller

Ejecución rápida

```
# Terminal 1
cd microservicio-productos && ./mvnw spring-boot:run     # :8081
# Terminal 2
cd microservicio-pedidos && ./mvnw spring-boot:run       # :8082

curl http://localhost:8082/pedidos/1
curl http://localhost:8082/actuator/metrics/http.server.requests
```

El script docs/scripts/ejecutar-experimentos.ps1 repite los experimentos completos y regraba las evidencias de texto en docs/evidencias/script/.